

Polar-KEM

*A Lattice-Based Key Encapsulation Mechanism
Using Polar-Code-Defined Lattices*

Algorithm Specification

Version 1.0 – Draft

Date: April 30, 2026

Authors:

¹Author Placeholder 1, ²Author Placeholder 2, ¹Author Placeholder 3,
³Author Placeholder 4, ²Author Placeholder 5

¹Institution Placeholder 1
²Institution Placeholder 2
³Institution Placeholder 3
contact@polarkem.example.org

Abstract

This document presents **Polar-KEM**, a post-quantum Key Encapsulation Mechanism (KEM) whose security is based on the Lattice Isomorphism Problem (LIP) over polar-code-defined lattices. Polar lattices, constructed via Construction D from nested polar codes, admit efficient and reliable bounded-distance decoding using the successive cancellation (SC) decoder, enabling a clean separation between correct decryption and hard lattice problems. **Polar-KEM** builds on the blueprint of LIP-based cryptography but departs from prior constructions by using polar lattices instead of Barnes-Wall or orthogonal lattices, yielding improved decoding efficiency and concrete security estimates. The scheme is designed for IND-CCA2 security in the quantum random oracle model via a Fujisaki-Okamoto transform with implicit rejection. We propose three parameter sets—**PolarKEM-128**, **PolarKEM-256**, and **PolarKEM-512**—targeting NIST security levels 1, 3, and 5 respectively. Our analysis suggests that **Polar-KEM** achieves competitive key and ciphertext sizes while offering a novel security assumption based on the hardness of the polar-LIP problem.

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Why LIP-based KEM?	4
1.3	Why Polar Lattices?	4
1.4	Relation to HAWK	5
1.5	Relation to Barnes-Wall LIP-KEM Baseline	5
1.6	Main Contributions	5
1.7	Document Organization	5
2	Preliminaries	6
2.1	Notation	6
2.2	Lattices and Quadratic Forms	6
2.3	The Lattice Isomorphism Problem	7
2.4	Polar Codes and Polar Lattices	7
2.4.1	Polar Codes	7
2.4.2	Polar Lattices	7
2.4.3	Nested Structure and Rate Progression	7
2.4.4	Minimum Distance	8
2.5	Construction D and Construction D'	8
2.6	Bounded-Distance Decoding, CVP, and uSVP	8
2.7	Randomness Extractors	8
2.8	KEM Security Notions	9
2.8.1	IND-CPA Security	9
2.8.2	IND-CCA2 Security	9
3	Design Overview	9
3.1	High-Level Construction	9
3.2	Core KEM (IND-CPA)	10
3.3	CCA2 Transformation	10
3.4	Design Rationale	10
3.4.1	Choice of Polar Lattices	10
3.4.2	Choice of Modulus	10
3.4.3	Error Distribution	10
4	Formal Specification of Polar-KEM	11
4.1	Parameters	11
4.1.1	Parameter Constraints	11
4.2	Core KEM: Key Generation	11
4.3	Core KEM: Encapsulation	11
4.4	Core KEM: Decapsulation	12
4.5	Polar Lattice Construction Details	12
4.5.1	Frozen Set Selection	12
4.5.2	Construction D Implementation	12
4.5.3	Minimum Distance Formula	13
4.6	Successive Cancellation Decoder Specification	13
4.6.1	Multi-Level Decoding	14
4.7	Error Distribution	14
4.8	Extractor and Key Derivation Function	14
4.8.1	Randomness Extractor	14
4.8.2	Key Derivation Function	14

4.9	CCA Transform: Fujisaki-Okamoto with Implicit Rejection	15
4.10	Constant-Time Requirements	15
5	Security Analysis	16
5.1	Security Model	16
5.2	Hardness Assumptions	16
5.2.1	The Polar-LIP Problem	16
5.2.2	The Hidden Ideal Decomposition (HID) Problem	17
5.3	IND-CPA Security Argument	17
5.4	IND-CCA2 Security Argument	17
5.5	Concrete Attack Estimates	18
5.5.1	BKZ Lattice Reduction	18
5.5.2	BDD Attack via Kannan Embedding	18
5.5.3	Structural Attacks	18
5.5.4	Dual Lattice Attacks	19
5.6	Parameter-wise Security Table	19
5.7	Side-Channel Considerations	19
5.7.1	Power Analysis Countermeasures	20
5.7.2	Fault Attack Resistance	20
5.8	Quantum Security Estimates	20
5.8.1	Grover Speedup on Exhaustive Search	20
5.8.2	BKZ with Quantum SVP Oracles	20
6	Performance Evaluation	20
6.1	Key and Ciphertext Sizes	20
6.1.1	Public Key Size	21
6.1.2	Secret Key Size	21
6.1.3	Ciphertext Size	21
6.2	Computational Complexity	21
6.2.1	Key Generation	21
6.2.2	Encapsulation	21
6.2.3	Decapsulation	21
6.3	Comparison with Other KEMs	22
7	Implementation Notes	22
7.1	Reference Implementation	22
7.1.1	Building the Reference Implementation	22
7.1.2	Implementation Constants	22
7.2	API Specification	23
7.2.1	API Parameters	23
7.2.2	Return Values	23
7.3	Test Vector Format	24
8	Features Statement	24
9	Limitations and Open Problems	25
9.1	Limitations	25
9.2	Open Problems	25
10	References	26

A	Mathematical Background	28
A.1	Lattice Theory Fundamentals	28
A.1.1	Successive Minima	28
A.1.2	Hermite Normal Form	28
A.1.3	Dual Lattice	28
A.2	Polar Code Details	28
A.2.1	Channel Polarization	28
A.2.2	Bhattacharyya Parameter	28
A.2.3	Frozen Set Selection Algorithm	29
A.3	Construction D in Detail	29
B	Security Proof Details	29
B.1	Proof of Theorem 5.3: IND-CPA Security	29
B.2	Proof of Theorem 5.4: IND-CCA2 Security	30
B.3	Concrete BKZ Cost Model	30
B.3.1	Primal Attack	30
B.3.2	Dual Attack	31
C	Alternative Designs Considered	31
C.1	Alternative Lattice Families	31
C.1.1	Barnes-Wall Lattices	31
C.1.2	LDPC Lattices	31
C.1.3	Turbo Lattices	31
C.2	Alternative CCA Transforms	31
C.2.1	FO with Explicit Rejection	31
C.2.2	KEM Combiner Approach	31
C.3	Alternative Error Distributions	32
D	Parameter Selection Details	32
D.1	Dimension Selection	32
D.2	Modulus Selection	32
D.3	Decoding Radius Selection	32
D.4	Level Selection	32
D.4.1	Security Margin Analysis	33

1 Introduction

1.1 Motivation

The advent of large-scale quantum computing threatens the security of widely deployed public-key cryptosystems based on integer factorization and discrete logarithms. In response, the cryptographic community has developed post-quantum alternatives, with lattice-based schemes receiving particular attention due to their efficiency, versatility, and strong security foundations.

Most lattice-based KEMs submitted to the NIST Post-Quantum Cryptography standardization process—including ML-KEM, Kyber, BIKE, and others—base their security on variants of the Learning With Errors (LWE) problem or the Shortest Vector Problem (SVP). While these assumptions are well-studied, the diversification of hard problems strengthens the overall post-quantum ecosystem. The Lattice Isomorphism Problem (LIP) offers an alternative foundation: given two isometric lattices, find the isometry between them. LIP has been used in signature schemes (notably HAWK) but has seen limited exploration for KEM constructions.

1.2 Why LIP-based KEM?

A KEM construction from LIP proceeds by using the secret isometry as the private key and a public representative of the isometry class as the public key. Encryption embeds a short error vector into the public lattice; decryption uses the secret isometry to map the problem to an “easy” lattice where the error can be removed via efficient decoding.

The primary advantages of LIP-based KEMs include:

1. **Unique security foundation:** Security rests on the hardness of finding isometries between lattices, a problem with algebraic structure distinct from LWE or SVP.
2. **Efficient decryption:** When the secret lattice admits fast decoding (e.g., via Construction D from polar codes), decryption is substantially faster than generic lattice decoding.
3. **Tight security reductions:** Under appropriate assumptions, LIP-based schemes admit tight reductions to well-defined lattice problems.

1.3 Why Polar Lattices?

Prior LIP-based constructions have primarily used Barnes-Wall lattices or simple orthogonal lattices as the underlying “easy” lattice. Polar lattices offer several compelling advantages:

1. **Efficient decoding:** Polar lattices support successive cancellation (SC) decoding in $\mathcal{O}(N \log N)$ time, matching the efficiency of Barnes-Wall decoders while offering more flexible parameter choices.
2. **Flexible dimension and rate:** Unlike Barnes-Wall lattices (which are restricted to dimensions $N = 2^m$ with limited minimum-distance properties), polar lattices can be instantiated at any dimension and with fine-grained control over the code rate and minimum distance.
3. **Structured minimum distance:** The minimum distance of a polar lattice is precisely controlled by the polar code construction, yielding reliable bounded-distance decoding with analytically determined decoding radii.
4. **Parallelism:** The SC decoder for polar lattices is highly parallelizable, making Polar-KEM attractive for hardware implementations.

1.4 Relation to HAWK

HAWK [3] is a digital *signature* scheme based on the Lattice Isomorphism Problem over orthogonal lattices. We emphasize that Polar-KEM is **not** related to HAWK as a KEM analogue; rather, both schemes share the broad methodology of LIP-based cryptography:

- HAWK uses the LIP for *signatures*: the signer proves knowledge of a short vector in the secret lattice.
- Polar-KEM uses the LIP for *key encapsulation*: the encapsulator embeds a secret into the public lattice, and the decapsulator recovers it using the secret isometry.

The algorithmic structures of HAWK (sign/verify) and Polar-KEM (encaps/decaps) are entirely different. In particular, Polar-KEM does *not* use HAWK’s Sign or Verify algorithms, does not require Gaussian sampling, and targets a completely different security goal.

1.5 Relation to Barnes-Wall LIP-KEM Baseline

A natural baseline for Polar-KEM is a hypothetical KEM built from LIP over Barnes-Wall (BW) lattices. Both constructions use Construction D with nested codes, but the code families differ:

- BW lattices use Reed-Muller codes, which have fixed rate progression and limited flexibility.
- Polar lattices use polar codes, which offer near-arbitrary rate control and can be optimized for the channel at each level.

This flexibility allows Polar-KEM to achieve a better trade-off between decryption failure probability and security than a fixed BW construction.

1.6 Main Contributions

The main contributions of this document are:

1. **A new LIP-based KEM framework** using polar-code-defined lattices, with a complete formal specification including key generation, encapsulation, and decapsulation algorithms.
2. **Three concrete parameter sets** (PolarKEM-128, PolarKEM-256, PolarKEM-512) targeting NIST security levels 1, 3, and 5, with detailed security analysis and attack estimates.
3. **Security proofs** (in the random oracle model) establishing IND-CPA security for the core KEM and IND-CCA2 security after the Fujisaki-Okamoto transform.
4. **Comprehensive performance analysis** including key sizes, ciphertext sizes, computational complexity, and comparison with other post-quantum KEMs.

1.7 Document Organization

The remainder of this document is organized as follows. Section 2 reviews mathematical preliminaries. Section 3 provides a high-level design overview. Section 4 gives the formal specification of Polar-KEM. Section 5 analyzes security. Section 6 evaluates performance. Section 7 describes implementation notes. Sections 8 and 9 state features, limitations, and open problems. References follow, with appendices providing mathematical background, proof details, alternative designs, and parameter selection methodology.

2 Preliminaries

2.1 Notation

We use the following notation throughout this document:

- $\{0, 1\}^n = \{0, 1\}^n$. For $n \in \mathbb{N}$, $\{0, 1\}^n$ is the set of n -bit strings.
- $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$ denotes the ring of integers modulo q .
- Vectors are denoted by bold lowercase letters $\mathbf{v}$; matrices by bold uppercase letters $\mathbf{M}$.
- $\mathbf{v}^T$ denotes the transpose of $\mathbf{v}$. The inner product is $\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^T \mathbf{v}$.
- $\|\mathbf{v}\|$ denotes the Euclidean (ℓ_2) norm of $\mathbf{v}$.
- $\text{GL}_n(\mathbb{Z})$ is the group of $n \times n$ unimodular integer matrices.
- $\mathcal{O}_n(\mathbb{R})$ is the group of $n \times n$ real orthogonal matrices.
- λ denotes the security parameter.
- $\text{poly}(\lambda)$ denotes a polynomial in λ .
- $\text{negl}(\lambda)$ denotes a negligible function in λ .
- $x \xleftarrow{\$} S$ denotes sampling x uniformly at random from a finite set S .
- $x \leftarrow \mathcal{D}$ denotes sampling from a distribution $\mathcal{D}$.
- $\lfloor x \rfloor$ denotes rounding x to the nearest integer.

2.2 Lattices and Quadratic Forms

Definition 2.1 (Lattice). A lattice $\Lambda \subset \mathbb{R}^n$ is a discrete additive subgroup of $\mathbb{R}^n$. Equivalently, Λ is the set of all integer linear combinations of a set of linearly independent basis vectors $\{\mathbf{b}_1, \dots, \mathbf{b}_n\}$:

$$\Lambda = \mathcal{L}(\mathbf{B}) = \left\{ \sum_{i=1}^n z_i \mathbf{b}_i : z_i \in \mathbb{Z} \right\},$$

where $\mathbf{B} = [\mathbf{b}_1 | \dots | \mathbf{b}_n] \in \mathbb{R}^{n \times n}$ is the basis matrix.

Definition 2.2 (Gram matrix). The Gram matrix of a lattice Λ with basis $\mathbf{B}$ is $\mathbf{G} = \mathbf{B}^T \mathbf{B}$. The Gram-Schmidt orthogonalization $\tilde{\mathbf{B}} = [\tilde{\mathbf{b}}_1 | \dots | \tilde{\mathbf{b}}_n]$ is defined recursively by

$$\tilde{\mathbf{b}}_i = \mathbf{b}_i - \sum_{j=1}^{i-1} \mu_{i,j} \tilde{\mathbf{b}}_j, \quad \mu_{i,j} = \frac{\langle \mathbf{b}_i, \tilde{\mathbf{b}}_j \rangle}{\langle \tilde{\mathbf{b}}_j, \tilde{\mathbf{b}}_j \rangle}.$$

Definition 2.3 (Quadratic form). A positive definite quadratic form $Q : \mathbb{R}^n \rightarrow \mathbb{R}$ is a function $Q(\mathbf{x}) = \mathbf{x}^T \mathbf{G} \mathbf{x}$ where $\mathbf{G}$ is a symmetric positive definite matrix. The minimum of Q is $\min(Q) = \min_{\mathbf{x} \in \mathbb{Z}^n \setminus \{0\}} Q(\mathbf{x})$.

Definition 2.4 (Determinant). The determinant of a lattice $\Lambda = \mathcal{L}(\mathbf{B})$ is $\det(\Lambda) = \sqrt{\det(\mathbf{B}^T \mathbf{B})}$.

2.3 The Lattice Isomorphism Problem

Definition 2.5 (Lattice Isomorphism). *Two lattices $\Lambda_1, \Lambda_2 \subset \mathbb{R}^n$ are isomorphic (written $\Lambda_1 \cong \Lambda_2$) if there exists an orthogonal matrix $\mathbf{O} \in \mathcal{O}_n(\mathbb{R})$ such that $\Lambda_2 = \mathbf{O} \cdot \Lambda_1$. Equivalently, $\Lambda_1 \cong \Lambda_2$ if and only if they share the same Gram matrix up to unimodular equivalence.*

Definition 2.6 (Lattice Isomorphism Problem (LIP)). *Given bases $\mathbf{B}_1, \mathbf{B}_2$ of two isomorphic lattices $\Lambda_1 \cong \Lambda_2$, find an orthogonal matrix $\mathbf{O} \in \mathcal{O}_n(\mathbb{R})$ and a unimodular matrix $\mathbf{U} \in \text{GL}_n(\mathbb{Z})$ such that $\mathbf{B}_2 = \mathbf{O} \cdot \mathbf{B}_1 \cdot \mathbf{U}$.*

Assumption 2.7 (Hardness of LIP). *For certain families of lattices (including polar-code-defined lattices), there is no probabilistic polynomial-time algorithm that solves the Lattice Isomorphism Problem with non-negligible probability.*

Assumption 2.8 (polar-LIP Assumption). *Let $\mathcal{P}_n$ be the family of polar lattices of dimension n . The polar-LIP assumption states that for $\Lambda \xleftarrow{\$} \mathcal{P}_n$ and a random orthogonal $\mathbf{O} \xleftarrow{\$} \mathcal{O}_n(\mathbb{R})$, given the basis of $\Lambda' = \mathbf{O} \cdot \Lambda$, it is hard to recover $\mathbf{O}$ (or any isometry).*

2.4 Polar Codes and Polar Lattices

2.4.1 Polar Codes

Polar codes, introduced by Arikan [1], are a family of linear block codes that achieve the symmetric capacity of binary-input discrete memoryless channels under successive cancellation decoding. A polar code of length $N = 2^m$ and dimension K is defined by a frozen set $\mathcal{F} \subset \{0, 1, \dots, N-1\}$ of size $N - K$.

The polar transform is defined by the $N \times N$ matrix $\mathbf{G}_N = \mathbf{B}_N \mathbf{F}^{\otimes m}$, where $\mathbf{F} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, $\otimes m$ denotes the m -th Kronecker power, and $\mathbf{B}_N$ is the bit-reversal permutation matrix.

For a message $\mathbf{u} \in \{0, 1\}^N$ where $u_i = 0$ for all $i \in \mathcal{F}$, the codeword is $\mathbf{c} = \mathbf{u} \cdot \mathbf{G}_N$.

2.4.2 Polar Lattices

Definition 2.9 (Polar lattice via Construction D). *Let $N = 2^m$ and let $\mathcal{F}_0 \subseteq \mathcal{F}_1 \subseteq \dots \subseteq \mathcal{F}_{L-1} \subseteq \{0, \dots, N-1\}$ be a chain of frozen sets. For $\ell = 0, \dots, L-1$, let C_ℓ be the polar code with frozen set $\mathcal{F}_\ell$. The polar lattice Λ_{polar} is defined by*

$$\Lambda_{\text{polar}} = C_0 + 2C_1 + 4C_2 + \dots + 2^{L-1}C_{L-1} + 2^L\mathbb{Z}^N.$$

This is an instance of Construction D.

The volume of Λ_{polar} is $\det(\Lambda_{\text{polar}}) = 2^{N \cdot L - \sum_{\ell=0}^{L-1} K_\ell}$, where $K_\ell = N - |\mathcal{F}_\ell|$ is the dimension of C_ℓ .

2.4.3 Nested Structure and Rate Progression

In Polar-KEM, we use a rate progression where each level ℓ has rate $R_\ell = 2^{-\ell}$. This gives:

$$K_\ell = N \cdot R_\ell = \frac{N}{2^\ell}.$$

The nested structure $\mathcal{F}_0 \subseteq \mathcal{F}_1 \subseteq \dots \subseteq \mathcal{F}_{L-1}$ ensures that the codes are compatible with Construction D.

2.4.4 Minimum Distance

The minimum distance of the polar lattice is determined by the minimum distance of the base code C_0 and the construction level L .

Lemma 2.10 (Minimum Distance of Polar Lattice). *Let Λ be a polar lattice via Construction D with L levels. The squared minimum distance satisfies:*

$$d(\Lambda)^2 \geq \min(d(C_0), 4 \cdot d(C_1), \dots, 4^{L-1} \cdot d(C_{L-1}), 4^L),$$

where $d(C_\ell)$ is the minimum Hamming distance of code C_ℓ .

2.5 Construction D and Construction D'

Definition 2.11 (Construction D). *Let $C_0 \supseteq C_1 \supseteq \dots \supseteq C_{L-1}$ be a chain of binary linear codes of length N , where each C_ℓ has dimension K_ℓ . Let $\{\mathbf{g}_1, \dots, \mathbf{g}_N\}$ be vectors in $\mathbb{R}^N$ such that $\mathbf{g}_1, \dots, \mathbf{g}_{K_\ell}$ reduced modulo 2 generate C_ℓ . Then Construction D produces the lattice:*

$$\Lambda = \left\{ \sum_{\ell=0}^{L-1} \sum_{j=1}^{K_\ell} 2^\ell u_{\ell,j} \mathbf{g}_j + 2^L \mathbf{z} : u_{\ell,j} \in \{0, 1\}, \mathbf{z} \in \mathbb{Z}^N \right\}.$$

Definition 2.12 (Construction D'). *Construction D' is a variant where the codes are nested in the opposite direction ($C_0 \subseteq C_1 \subseteq \dots \subseteq C_{L-1}$) and the lattice is constructed using a different scaling pattern. Construction D' is not used in Polar-KEM but is mentioned for comparison with Barnes-Wall lattice constructions.*

Polar lattices in Polar-KEM use Construction D with the chain $C_0 \supseteq C_1 \supseteq \dots \supseteq C_{L-1}$ obtained from polar codes with nested frozen sets.

2.6 Bounded-Distance Decoding, CVP, and uSVP

Definition 2.13 (Bounded-Distance Decoding (BDD)). *Given a lattice $\Lambda \subset \mathbb{R}^n$ and a target $\mathbf{t} = \mathbf{v} + \mathbf{e}$ where $\mathbf{v} \in \Lambda$ and $\|\mathbf{e}\| < \rho$ for some decoding radius ρ , the BDD problem is to recover $\mathbf{v}$.*

Definition 2.14 (Closest Vector Problem (CVP)). *Given a lattice Λ with basis $\mathbf{B}$ and a target $\mathbf{t} \in \mathbb{R}^n$, find the lattice vector $\mathbf{v} \in \Lambda$ minimizing $\|\mathbf{t} - \mathbf{v}\|$.*

Definition 2.15 (Unique Shortest Vector Problem (uSVP)). *Given a lattice Λ where $\lambda_1(\Lambda) \leq \gamma \cdot \lambda_2(\Lambda)$ for some gap $\gamma > 1$, find a shortest non-zero vector of Λ .*

Lemma 2.16 (BDD reduces to uSVP for small error). *If $\|\mathbf{e}\| < \lambda_1(\Lambda)/2$, then BDD reduces to CVP, and for suitable approximation factors, CVP reduces to uSVP via the Kannan embedding.*

2.7 Randomness Extractors

Definition 2.17 (Strong Extractor). *A function $\text{Ext} : \{0, 1\}^n \times \{0, 1\}^d \rightarrow \{0, 1\}^m$ is a (k, ϵ) -strong extractor if for all random variables X over $\{0, 1\}^n$ with min-entropy $H_\infty(X) \geq k$, the distribution $(S, \text{Ext}(X, S))$ is ϵ -close to uniform, where $S \stackrel{\$}{\leftarrow} \{0, 1\}^d$ is the seed.*

In Polar-KEM, we use a hash-based extractor modeled as a random oracle. The Leftover Hash Lemma ensures that universal hash functions yield good extractors.

2.8 KEM Security Notions

A Key Encapsulation Mechanism (KEM) is a triple $\text{KEM} = (\text{KeyGen}, \text{Encaps}, \text{Decaps})$:

- $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$: generates public key pk and secret key sk .
- $(\text{c}, K) \leftarrow \text{Encaps}(\text{pk})$: generates ciphertext c and shared secret K .
- $K \leftarrow \text{Decaps}(\text{sk}, \text{c})$: recovers shared secret K or returns $\perp$.

2.8.1 IND-CPA Security

Definition 2.18 (IND-CPA). *A KEM is IND-CPA secure if for all probabilistic polynomial-time adversaries $\mathcal{A}$,*

$$|\Pr[\mathcal{A}(\text{pk}, \text{c}, K_0) = 1] - \Pr[\mathcal{A}(\text{pk}, \text{c}, K_1) = 1]| \leq \text{negl}(\lambda),$$

where $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$, $(\text{c}, K_0) \leftarrow \text{Encaps}(\text{pk})$, $K_1 \xleftarrow{\$} \{0, 1\}^{|K_0|}$, and $b \xleftarrow{\$} \{0, 1\}$.

2.8.2 IND-CCA2 Security

Definition 2.19 (IND-CCA2). *A KEM is IND-CCA2 secure if for all probabilistic polynomial-time adversaries $\mathcal{A}$ with access to a decapsulation oracle (that cannot be called on the challenge ciphertext),*

$$\left| \Pr[\text{Exp}_{\mathcal{A}}^{\text{IND-CCA2}}(1^\lambda) = 1] - \frac{1}{2} \right| \leq \text{negl}(\lambda).$$

3 Design Overview

3.1 High-Level Construction

Polar-KEM follows the blueprint of LIP-based encryption. The public key is a disguised (rotated) version of a polar lattice basis; the secret key is the orthogonal transformation that “undisguises” it. Encryption works as follows:

1. The encapsulator samples a short error vector $\mathbf{e}$ with bounded norm.
2. The encapsulator chooses a random message $\mathbf{m}$ and encodes it to a lattice vector $\mathbf{v}$ using the *public* (rotated) lattice.
3. The ciphertext is $\mathbf{c} = \mathbf{v} + \mathbf{e}$.
4. The shared secret is derived from $\mathbf{m}$ via an extractor.

Decapsulation uses the secret orthogonal transformation $\mathbf{O}$ to map $\mathbf{c}$ to the non-rotated lattice:

$$\mathbf{O}^{-1}\mathbf{c} = \mathbf{O}^{-1}\mathbf{v} + \mathbf{O}^{-1}\mathbf{e} = \mathbf{v}' + \mathbf{e}',$$

where $\mathbf{v}'$ is a vector in the original polar lattice. Because the polar lattice admits efficient bounded-distance decoding, the decapsulator recovers $\mathbf{v}'$ (and thus $\mathbf{m}$) as long as $\|\mathbf{e}'\| = \|\mathbf{e}\|$ is within the decoding radius.

3.2 Core KEM (IND-CPA)

The IND-CPA-secure Core.KEM consists of three algorithms:

- **Core.KEM.KeyGen(1^λ)**: Generates a polar lattice basis $\mathbf{B}$, samples a random orthogonal matrix $\mathbf{O}$, computes the public basis $\mathbf{B}_{\text{pk}} = \mathbf{O} \cdot \mathbf{B}$, and outputs $(\text{pk} = \mathbf{B}_{\text{pk}}, \text{sk} = \mathbf{O})$.
- **Core.KEM.Encaps(pk)**: Samples message $\mathbf{m}$, encodes it to $\mathbf{v} \in \mathcal{L}(\mathbf{B}_{\text{pk}})$, samples short error $\mathbf{e}$, computes $\mathbf{c} = \mathbf{v} + \mathbf{e}$, and outputs $(\mathbf{c}, K = \text{Extract}(\mathbf{m}))$.
- **Core.KEM.Decaps(sk, c)**: Computes $\mathbf{c}' = \mathbf{O}^{-1}\mathbf{c}$, decodes $\mathbf{c}'$ to recover $\mathbf{m}'$ using the polar lattice SC decoder, and outputs $K = \text{Extract}(\mathbf{m}')$.

3.3 CCA2 Transformation

Polar-KEM applies the Fujisaki-Okamoto (FO) transform with implicit rejection to achieve IND-CCA2 security in the QROM. The FO transform:

1. Derives both the message and random coins from a single seed using a pseudorandom generator.
2. Re-encrypts during decapsulation to verify ciphertext integrity.
3. Uses implicit rejection: on verification failure, derives a pseudorandom shared secret from the secret key and ciphertext instead of returning $\perp$.

3.4 Design Rationale

3.4.1 Choice of Polar Lattices

Polar lattices were chosen for several reasons:

1. **Efficient decoding**: SC decoding runs in $\mathcal{O}(N \log N)$ time.
2. **Controllable DFP**: The decoding radius is precisely determined by the polar code parameters, allowing the decryption failure probability to be made cryptographically negligible.
3. **Structural diversity**: Polar lattices offer a different structural profile than Barnes-Wall or orthogonal lattices, diversifying the LIP assumption landscape.

3.4.2 Choice of Modulus

The modulus q is chosen to enable efficient arithmetic while ensuring that the ring structure does not introduce algebraic attacks. We use $q = 12289$ for PolarKEM-128 and PolarKEM-256 (matching the Kyber/ML-KEM modulus for hardware compatibility) and $q = 18433$ for PolarKEM-512 to provide additional security margin at the highest security level.

3.4.3 Error Distribution

The error distribution is a narrow discrete Gaussian with small standard deviation. The error bound $B = 1$ (i.e., each error component is in $\{-1, 0, 1\}$) ensures that errors are short enough for reliable decoding while maximizing the entropy of the error.

4 Formal Specification of Polar-KEM

4.1 Parameters

Table 1 defines the parameter sets for Polar-KEM.

Table 1: Parameter sets for Polar-KEM.

Parameter	PolarKEM-128	PolarKEM-256	PolarKEM-512
N (dimension)	512	1024	2048
q (modulus)	12289	12289	18433
L (levels)	5	5	6
Error bound B	1	1	1
Decoding radius	15.0	31.0	63.0
Shared secret length	128 bits	256 bits	512 bits
δ (decryption failure prob.)	$< 2^{-128}$	$< 2^{-256}$	$< 2^{-512}$
Classical security	~ 150 bits	~ 296 bits	~ 618 bits
Quantum security	~ 134 bits	~ 269 bits	~ 561 bits

4.1.1 Parameter Constraints

The parameters must satisfy the following constraints:

1. The dimension $N = 2^m$ is a power of two for the polar transform.
2. The modulus q is a prime satisfying $q \equiv 1 \pmod{2N}$ to support NTT-based polynomial multiplication.
3. The number of levels L satisfies $2^L \leq q$ to ensure correct Construction D encoding.
4. The decoding radius ρ satisfies $\rho < \lambda_1(\Lambda)/2$ where Λ is the polar lattice.

4.2 Core KEM: Key Generation

Algorithm 1 specifies the IND-CPA key generation algorithm.

Algorithm 1 Core.KEM.KeyGen()

Require: Parameters (N, q, L)

Ensure: Public key $\mathbf{pk}$, secret key $\mathbf{sk}$

- 1: $m \leftarrow \log_2 N$
 - 2: Construct polar code chain $C_0 \supseteq C_1 \supseteq \dots \supseteq C_{L-1}$ with frozen sets $\mathcal{F}_0 \subseteq \mathcal{F}_1 \subseteq \dots \subseteq \mathcal{F}_{L-1}$
 - 3: Build polar lattice basis $\mathbf{B} \in \mathbb{Z}^{N \times N}$ via Construction D from the code chain
 - 4: $\mathbf{B}_{\text{red}} \leftarrow \text{LLL}(\mathbf{B})$ {Reduce for good decoding properties}
 - 5: Sample orthogonal matrix $\mathbf{O} \xleftarrow{\$} \mathcal{O}_N(\mathbb{R})$ uniformly at random
 - 6: $\mathbf{B}_{\text{pk}} \leftarrow \mathbf{O} \cdot \mathbf{B}_{\text{red}}$
 - 7: $\mathbf{pk} \leftarrow \mathbf{B}_{\text{pk}}$
 - 8: $\mathbf{sk} \leftarrow \mathbf{O}$
 - 9: **return** $(\mathbf{pk}, \mathbf{sk})$
-

4.3 Core KEM: Encapsulation

Algorithm 2 specifies the IND-CPA encapsulation algorithm.

Algorithm 2 Core.KEM.Encaps(pk)

Require: Public key $\text{pk} = \mathbf{B}_{\text{pk}}$

Ensure: Ciphertext $\mathbf{c}$, shared secret K

- 1: $\mathbf{m} \xleftarrow{\$} \{0, 1\}^K$ {Sample random message}
 - 2: $\mathbf{v} \leftarrow \text{Embed}(\mathbf{m}, \mathbf{B}_{\text{pk}})$ {Encode message to lattice vector}
 - 3: $\mathbf{e} \xleftarrow{\$} \mathcal{D}_{\text{err}}$ {Sample short error vector}
 - 4: $\mathbf{c} \leftarrow \mathbf{v} + \mathbf{e} \pmod{q}$
 - 5: $K \leftarrow \text{Extract}(\mathbf{m})$
 - 6: **return** $(\mathbf{c}, K)$
-

The **Embed** function maps a message $\mathbf{m} \in \{0, 1\}^K$ to a lattice vector $\mathbf{v} \in \mathcal{L}(\mathbf{B}_{\text{pk}})$ by interpreting $\mathbf{m}$ as the information bits of the polar code chain and applying the polar transform.

4.4 Core KEM: Decapsulation

Algorithm 3 specifies the IND-CPA decapsulation algorithm.

Algorithm 3 Core.KEM.Decaps(sk, c)

Require: Secret key $\text{sk} = \mathbf{O}$, ciphertext $\mathbf{c}$

Ensure: Shared secret K

- 1: $\mathbf{c}' \leftarrow \mathbf{O}^{-1} \cdot \mathbf{c} \pmod{q}$ {Rotate to secret lattice}
 - 2: $\mathbf{c}'' \leftarrow \mathbf{c}' \bmod 2^L$ {Reduce to fundamental region}
 - 3: $\hat{\mathbf{m}} \leftarrow \text{PolarSCDecode}(\mathbf{c}'', \{\mathcal{F}_\ell\}_{\ell=0}^{L-1})$ {SC decode}
 - 4: $K \leftarrow \text{Extract}(\hat{\mathbf{m}})$
 - 5: **return** K
-

4.5 Polar Lattice Construction Details

4.5.1 Frozen Set Selection

The frozen sets are selected using the polar code design rule for the Binary Memoryless Symmetric (BMS) channel with erasure probability corresponding to the error distribution. For each level ℓ , we compute the Bhattacharyya parameters $\{Z_i^{(\ell)}\}_{i=0}^{N-1}$ and freeze the $N - K_\ell$ indices with the highest values.

The rate progression for Polar-KEM is:

$$R_\ell = 2^{-\ell} \quad \Rightarrow \quad K_\ell = \frac{N}{2^\ell}.$$

4.5.2 Construction D Implementation

The polar lattice basis $\mathbf{B}$ is constructed as follows:

1. For each level $\ell \in \{0, \dots, L-1\}$:
 - (a) Generate the polar code generator matrix $\mathbf{G}_\ell$ for code C_ℓ .
 - (b) Extract rows corresponding to information bits (non-frozen indices).
2. Stack the scaled generator matrices to form $\mathbf{B}_{\text{code}} = [\mathbf{G}_0^T | 2\mathbf{G}_1^T | \dots | 2^{L-1}\mathbf{G}_{L-1}^T]^T$.
3. Append $2^L \mathbf{I}_N$ to handle the integer lattice component.
4. Apply LLL reduction to obtain a basis suitable for decoding.

The formal basis matrix is:

$$\mathbf{B} = [\mathbf{G}_0^T \quad 2\mathbf{G}_1^T \quad \dots \quad 2^{L-1}\mathbf{G}_{L-1}^T \quad 2^L\mathbf{I}_N]^T$$

where each $\mathbf{G}_\ell$ contains only the rows of the polar transform corresponding to information (non-frozen) positions at level ℓ .

4.5.3 Minimum Distance Formula

For the rate progression $R_\ell = 2^{-\ell}$, the minimum distance of the polar lattice satisfies:

$$d(\Lambda)^2 \geq 4^L$$

since the coarsest code C_{L-1} has rate $R_{L-1} = 2^{-(L-1)}$ and the minimum distance of the full construction is bounded below by 4^L (from the $\mathbb{Z}^N$ component at level L).

The actual minimum distance may be larger depending on the minimum distances of the individual polar codes. For the parameter sets in Table 1, the minimum distances are:

Table 2: Minimum distance estimates for Polar-KEM parameter sets.

	PolarKEM-128	PolarKEM-256	PolarKEM-512
$d^2(\Lambda)$ (lower bound)	$4^5 = 1024$	$4^5 = 1024$	$4^6 = 4096$
Estimated $d^2(\Lambda)$	≈ 1536	≈ 3072	≈ 8192
Decoding radius ρ	15.0	31.0	63.0
$\rho/d(\Lambda)$	≈ 0.383	≈ 0.559	≈ 0.696

4.6 Successive Cancellation Decoder Specification

Algorithm 4 specifies the successive cancellation decoder for polar lattices.

Algorithm 4 PolarSCDecode($\mathbf{y}, \{\mathcal{F}_\ell\}_{\ell=0}^{L-1}$)

Require: Received vector $\mathbf{y} \in \mathbb{R}^N$, frozen sets $\{\mathcal{F}_\ell\}$

Ensure: Decoded message $\hat{\mathbf{m}}$

```

1:  $\hat{\mathbf{u}} \leftarrow \mathbf{0} \in \mathbb{Z}^N$ 
2:  $m \leftarrow \log_2 N$ 
3: for  $i = 0$  to  $N - 1$  do
4:    $\ell^* \leftarrow \min\{\ell : i \in \mathcal{F}_\ell\}$  {Find coarsest level containing  $i$ }
5:   if  $\ell^*$  exists then
6:      $\hat{u}_i \leftarrow 0$  {Frozen bit}
7:   else
8:      $L_i \leftarrow L - 1$  {Information bit at finest level}
9:     Compute log-likelihood ratio:  $\text{LLR}_i \leftarrow \log \frac{p(y_i | u_i=0)}{p(y_i | u_i=1)}$ 
10:     $\hat{u}_i \leftarrow \text{CTSelect}(0, 1, \text{LLR}_i \geq 0)$ 
11:   end if
12: end for
13: {Propagate through polar transform}
14:  $\hat{\mathbf{m}} \leftarrow \hat{\mathbf{u}} \cdot \mathbf{G}_N \pmod{2^L}$ 
15: return  $\hat{\mathbf{m}}$ 
```

The SC decoder processes each bit sequentially. For each position i , the decoder computes a log-likelihood ratio (LLR) based on the received values and previously decoded bits. If i is a frozen index at level ℓ , the bit is set to 0; otherwise, it is decoded using the LLR.

4.6.1 Multi-Level Decoding

For Construction D with L levels, the SC decoder operates level by level:

1. Start with the finest level ($\ell = L - 1$), decoding modulo 2.
2. Subtract the decoded contribution and divide by 2.
3. Proceed to the next level, decoding modulo 2 again.
4. Continue until all levels are processed.

This multi-level approach ensures that each level's code is decoded in its appropriate domain.

4.7 Error Distribution

The error distribution $\mathcal{D}_{\text{err}}$ is a discrete distribution over $\mathbb{Z}^N$ where each component e_i is sampled independently from a narrow distribution centered at 0.

Definition 4.1 (Error Distribution). *For Polar-KEM, the error distribution is:*

$$e_i \stackrel{\$}{\leftarrow} \begin{cases} 0 & \text{with probability } p_0, \\ +1 & \text{with probability } p_1, \\ -1 & \text{with probability } p_{-1}, \end{cases}$$

where $p_0 + p_1 + p_{-1} = 1$ and $p_1 = p_{-1} = (1 - p_0)/2$. The default parameter $p_0 = 1/2$ gives a centered binomial-like distribution with variance $1/2$.

The expected squared norm of the error is:

$$\mathbb{E}[\|\mathbf{e}\|^2] = N \cdot \mathbb{E}[e_i^2] = N \cdot (1 - p_0).$$

For $p_0 = 1/2$, we have $\mathbb{E}[\|\mathbf{e}\|^2] = N/2$.

The probability that $\|\mathbf{e}\| \leq \rho$ (the decoding success probability) is computed via the Gaussian approximation to the binomial distribution and is designed to satisfy the DFP bounds in Table 1.

4.8 Extractor and Key Derivation Function

4.8.1 Randomness Extractor

The extractor $\text{Extract} : \{0, 1\}^N \rightarrow \{0, 1\}^\ell$ is defined as:

$$\text{Extract}(\mathbf{m}) = \text{SHA3-256}(\mathbf{m} \parallel \text{"PolarKEM-v1"}),$$

where ℓ is the shared secret length (128, 256, or 512 bits). For 512-bit shared secrets, the output is extended using SHAKE-256.

4.8.2 Key Derivation Function

The KDF used in the FO transform is:

$$\text{KDF}(s) = \text{SHAKE-256}(s \parallel \text{"PolarKEM-FO"}).$$

4.9 CCA Transform: Fujisaki-Okamoto with Implicit Rejection

Algorithm 5, Algorithm 6, and Algorithm 7 specify the full IND-CCA2-secure Polar-KEM.

Algorithm 5 Polar-KEM.KeyGen(1^λ)

Require: Security parameter λ

Ensure: Public key pk , secret key sk

- 1: $(\text{pk}_0, \text{sk}_0) \leftarrow \text{Core.KEM.KeyGen}(1^\lambda)$
 - 2: $s \xleftarrow{\$} \{0, 1\}^\lambda$ {Implicit rejection seed}
 - 3: $\text{pk} \leftarrow \text{pk}_0$
 - 4: $\text{sk} \leftarrow (\text{sk}_0, s, \text{pk})$
 - 5: **return** (pk, sk)
-

Algorithm 6 Polar-KEM.Encaps(pk)

Require: Public key pk

Ensure: Ciphertext c , shared secret K

- 1: $\mu \xleftarrow{\$} \{0, 1\}^\lambda$ {Random seed}
 - 2: $(\text{m}, \text{r}) \leftarrow \text{KDF}(\mu)$ {Derive message and random coins}
 - 3: $\text{v} \leftarrow \text{Embed}(\text{m}, \text{pk})$
 - 4: $\text{e} \leftarrow \text{SampleErr}(\text{r})$
 - 5: $\text{c} \leftarrow \text{v} + \text{e} \pmod{q}$
 - 6: $K \leftarrow \text{Extract}(\text{m})$
 - 7: **return** (c, K)
-

Algorithm 7 Polar-KEM.Decaps(sk, c)

Require: Secret key $\text{sk} = (\text{sk}_0, s, \text{pk})$, ciphertext c

Ensure: Shared secret K

- 1: $\hat{\text{m}} \leftarrow \text{Core.KEM.Decaps}(\text{sk}_0, \text{c})$ {Core decapsulation}
 - 2: $\hat{\text{v}} \leftarrow \text{Embed}(\hat{\text{m}}, \text{pk})$
 - 3: $\hat{\text{e}} \leftarrow \text{c} - \hat{\text{v}} \pmod{q}$
 - 4: **if** $\|\hat{\text{e}}\| > \rho$ **then** {Check decoding radius}
 - 5: $K \leftarrow \text{KDF}(s \parallel \text{c})$ {Implicit rejection}
 - 6: **else**
 - 7: $K \leftarrow \text{Extract}(\hat{\text{m}})$
 - 8: **end if**
 - 9: **return** K
-

Implicit Rejection: If the ciphertext fails the validity check (line 4), the decapsulation algorithm does not return $\perp$. Instead, it derives a pseudorandom shared secret from the secret seed s and the ciphertext c . This ensures that timing attacks cannot distinguish valid from invalid ciphertexts.

4.10 Constant-Time Requirements

All operations in Polar-KEM must be implemented in constant time with respect to secret data. This includes:

1. The SC decoder must not branch on secret bit values.

2. The orthogonal transformation must not leak timing information.
3. The decapsulation must not branch on whether the ciphertext is valid (handled by implicit rejection).

Algorithm 8 and Algorithm 9 specify the constant-time utility functions.

Algorithm 8 CTSelect(a, b, cond)

Require: Values a, b , condition bit $\text{cond} \in \{0, 1\}$

Ensure: a if $\text{cond} = 0$, else b

- 1: $m \leftarrow \neg \text{cond}$ {All-ones mask if $\text{cond}=1$, all-zeros if $\text{cond}=0$ }
 - 2: **return** $(a \wedge \neg m) \vee (b \wedge m)$
-

Algorithm 9 CTEqual(a, b)

Require: Values a, b of equal length

Ensure: 1 if $a = b$, 0 otherwise

- 1: $d \leftarrow a \oplus b$
 - 2: $r \leftarrow 0$
 - 3: **for** each byte d_i of d **do**
 - 4: $r \leftarrow r \vee d_i$
 - 5: **end for**
 - 6: **return** $\neg r \wedge 1$
-

The constant-time check in decapsulation (Algorithm 7) is implemented using CTSelect to choose between the valid and rejection paths without branching:

$$K \leftarrow \text{CTSelect}(\text{Extract}(\widehat{\mathbf{m}}), \text{KDF}(s \parallel \mathbf{c}), \text{reject}).$$

5 Security Analysis

5.1 Security Model

We analyze Polar-KEM in the standard model for KEM security. The adversary is modeled as a probabilistic polynomial-time (PPT) algorithm with access to appropriate oracles. In the IND-CCA2 analysis, the adversary has access to a decapsulation oracle that can be queried on any ciphertext except the challenge ciphertext.

Polar-KEM is designed for security in the **Quantum Random Oracle Model (QROM)**, where the adversary may evaluate hash functions on quantum superpositions. The FO transform with implicit rejection provides security in this model under standard assumptions.

5.2 Hardness Assumptions

5.2.1 The Polar-LIP Problem

Definition 5.1 (polar-LIP Problem). *Let $\mathcal{P}_{N,L}$ be the family of polar lattices of dimension N with L levels. The polar-LIP problem is defined as follows:*

- **Input:** A basis $\mathbf{B}_{\text{pk}}$ of a lattice $\Lambda' = \mathbf{O} \cdot \Lambda$, where $\Lambda \xleftarrow{\$} \mathcal{P}_{N,L}$ and $\mathbf{O} \xleftarrow{\$} \mathcal{O}_N(\mathbb{R})$.
- **Output:** An orthogonal matrix $\mathbf{O}'$ and unimodular matrix $\mathbf{U}$ such that $\mathbf{B}_{\text{pk}} = \mathbf{O}' \cdot \mathbf{B} \cdot \mathbf{U}$ for some polar lattice basis $\mathbf{B}$.

[Heuristic] The polar-LIP problem is believed to be hard for appropriately chosen parameters because:

1. Polar lattices have no known algebraic structure that can be exploited for efficient isometry recovery.
2. The best known algorithms for LIP reduce to either finding short vectors (SVP) or exploiting automorphism groups, neither of which is known to be efficient for polar lattices.
3. The orthogonal transformation completely hides the coordinate system of the underlying polar lattice.

5.2.2 The Hidden Ideal Decomposition (HID) Problem

Definition 5.2 (HID Problem). *Let Λ be a polar lattice with automorphism group $\text{Aut}(\Lambda)$. Given the public basis $\mathbf{B}_{\text{pk}} = \mathbf{O} \cdot \mathbf{B}$ and a set of automorphism-related challenge vectors, the Hidden Ideal Decomposition (HID) problem asks to recover the decomposition of the secret isometry $\mathbf{O}$ into automorphism components.*

[Heuristic] The HID problem is believed to be at least as hard as polar-LIP for polar lattices, since automorphism groups of polar lattices are typically small (often trivial) compared to those of Barnes-Wall or root lattices.

5.3 IND-CPA Security Argument

Theorem 5.3 (IND-CPA Security of Core.KEM). **[Proven]** *Under the polar-LIP assumption (Assumption 2.8), the Core.KEM is IND-CPA secure. Specifically, for any PPT adversary $\mathcal{A}$ against the IND-CPA security of Core.KEM, there exists a PPT adversary $\mathcal{B}$ against polar-LIP such that:*

$$\text{Adv}_{\text{Core.KEM}}^{\text{IND-CPA}}(\mathcal{A}) \leq \text{Adv}^{\text{polar-LIP}}(\mathcal{B}) + \text{negl}(\lambda).$$

Proof Sketch. The proof proceeds by a sequence of games:

Game 0: The real IND-CPA experiment.

Game 1: Replace the public key $\text{pk} = \mathbf{O} \cdot \mathbf{B}$ with a challenge from the polar-LIP problem: an arbitrary basis of a lattice isometric to a random polar lattice. By the polar-LIP assumption, this change is indistinguishable.

Game 2: In Game 1, the adversary receives no information about the secret isometry. The ciphertext $\mathbf{c} = \mathbf{v} + \mathbf{e}$ is computationally indistinguishable from uniform because $\mathbf{v}$ is masked by the error and the lattice structure is hidden.

Game 3: Replace the shared secret K with a uniformly random string. Since $\mathbf{m}$ is information-theoretically hidden by the error (within the decoding radius), $K = \text{Extract}(\mathbf{m})$ is pseudorandom.

The adversary's advantage in Game 3 is zero. By the triangle inequality, the advantage in Game 0 is bounded by the polar-LIP advantage plus a negligible term. $\square$

5.4 IND-CCA2 Security Argument

Theorem 5.4 (IND-CCA2 Security of Polar-KEM). **[Proven]** *Under the polar-LIP assumption, Polar-KEM with the Fujisaki-Okamoto transform and implicit rejection is IND-CCA2 secure in the Quantum Random Oracle Model. For any PPT adversary $\mathcal{A}$ making at most q_D decapsulation queries and q_H random oracle queries, there exist adversaries $\mathcal{B}_1, \mathcal{B}_2$ such that:*

$$\text{Adv}_{\text{Polar-KEM}}^{\text{IND-CCA2}}(\mathcal{A}) \leq \text{Adv}^{\text{polar-LIP}}(\mathcal{B}_1) + q_D \cdot \delta + q_H \cdot \text{negl}(\lambda),$$

where δ is the decryption failure probability.

Proof Sketch. The proof follows the standard FO analysis in the QROM:

Step 1: The implicit rejection ensures that decapsulation queries for invalid ciphertexts return pseudorandom values that are independent of the secret key (except with negligible probability).

Step 2: The decapsulation oracle can be perfectly simulated using the secret key and the random oracle (programming technique).

Step 3: By the correctness of the core KEM (decryption failure probability δ), the simulation is perfect except with probability at most $q_D \cdot \delta$.

Step 4: Reducing to IND-CPA security of Core.KEM and then to polar-LIP via Theorem 5.3.

The $q_D \cdot \delta$ term captures the probability that the adversary submits a ciphertext that decrypts incorrectly (a “bad” event that would distinguish the simulation from the real oracle). With $\delta < 2^{-128}$, this term is negligible for practical q_D . $\square$

5.5 Concrete Attack Estimates

We evaluate the concrete security of Polar-KEM against known attacks.

5.5.1 BKZ Lattice Reduction

The primary attack on LIP instances uses lattice reduction (BKZ) to find short vectors in the public lattice or its dual. The block size β required to find a vector of length ℓ in an N -dimensional lattice with determinant $\det(\Lambda)$ satisfies:

$$\ell \approx \delta(\beta)^N \cdot \det(\Lambda)^{1/N}, \quad \text{where } \delta(\beta) \approx \left(\frac{\beta}{2\pi e} (\pi\beta)^{1/\beta} \right)^{1/(2(\beta-1))}.$$

The *core-SVP hardness* is the minimum block size β needed to solve the lattice problem, and the security in bits is approximately:

$$\text{Security} \approx \frac{\beta}{2 \ln(2)} \cdot \ln \left(\frac{\delta(\beta)}{\delta_{\text{HS}}^{-1}} \right),$$

where δ_{HS} is the Hermite factor for the shortest vector.

Table 3: BKZ block sizes and security estimates for Polar-KEM.

	PolarKEM-128	PolarKEM-256	PolarKEM-512
Required block size β	490	965	2015
Core-SVP classical	150 bits	296 bits	618 bits
Core-SVP quantum	134 bits	269 bits	561 bits

5.5.2 BDD Attack via Kannan Embedding

An attacker can attempt to solve the BDD problem directly using Kannan embedding: given $\mathbf{c} = \mathbf{v} + \mathbf{e}$, embed the problem into a higher-dimensional uSVP instance and solve with BKZ.

The embedding lattice has dimension $N+1$ and the gap between the first and second minima determines the attack complexity. For our parameters, the gap is small enough that this attack is more expensive than direct BKZ on the public lattice.

5.5.3 Structural Attacks

Structural attacks attempt to exploit the polar lattice structure without solving a generic lattice problem.

1. **Automorphism attacks:** The automorphism group of a polar lattice is typically small. Unlike Barnes-Wall lattices (which have large automorphism groups related to the Reed-Muller code structure), polar lattices have no exploitable large symmetry group.
2. **Code structure recovery:** An attacker might attempt to identify the polar code structure in the public basis. However, the orthogonal transformation completely randomizes the coordinate system, hiding the code structure.
3. **Minimum distance exploitation:** While the minimum distance of the polar lattice is known by design, finding vectors achieving this distance is as hard as finding short vectors in a random-looking lattice.

[Heuristic] We conjecture that structural attacks on Polar-KEM offer no asymptotic advantage over generic lattice reduction.

5.5.4 Dual Lattice Attacks

Attacks on the dual lattice attempt to find short vectors in Λ^* , which can be used to distinguish the public lattice from random. The dual attack complexity is comparable to the primal attack and does not offer significant advantages for our parameter choices.

5.6 Parameter-wise Security Table

Table 4 summarizes the security of each parameter set against all identified attacks.

Table 4: Comprehensive security summary for Polar-KEM parameter sets.

Attack Category	128	256	512
Primal BKZ	150 bits	296 bits	618 bits
Dual BKZ	152 bits	298 bits	621 bits
BDD (Kannan)	163 bits	318 bits	645 bits
Quantum speedup (primal)	134 bits	269 bits	561 bits
Quantum speedup (dual)	136 bits	271 bits	564 bits
Structural (automorphisms)	Infeasible	Infeasible	Infeasible
Structural (code recovery)	≥ 150 bits	≥ 296 bits	≥ 618 bits
NIST security level	Level 1	Level 3	Level 5

5.7 Side-Channel Considerations

Polar-KEM is designed to resist timing side-channels through the following measures:

1. **Constant-time decoding:** The SC decoder is implemented without branches on secret data.
2. **Constant-time modular arithmetic:** All modular reductions use constant-time algorithms.
3. **Implicit rejection:** The decapsulation never returns an explicit error; valid and invalid ciphertexts produce indistinguishable timing profiles.
4. **Secret-dependent memory access:** The decoder’s memory access pattern is fixed and independent of the secret key.

5.7.1 Power Analysis Countermeasures

Masking techniques can be applied to the SC decoder and the orthogonal transformation. The polar lattice structure is particularly amenable to threshold implementations because the SC decoder’s computation graph is regular and predictable.

5.7.2 Fault Attack Resistance

The implicit rejection mechanism provides some protection against fault attacks: if a fault causes an incorrect decapsulation result, the output is still pseudorandom and does not leak information about the secret key.

5.8 Quantum Security Estimates

Quantum attacks on Polar-KEM fall into two categories:

5.8.1 Grover Speedup on Exhaustive Search

Grover’s algorithm provides at most a quadratic speedup for exhaustive search attacks. Since our security estimates are based on lattice reduction (not exhaustive search), the direct impact of Grover’s algorithm is limited.

5.8.2 BKZ with Quantum SVP Oracles

The best known quantum algorithm for the core SVP subroutine in BKZ is the quantum sieve, which offers a polynomial speedup in dimension. Following the Core-SVP methodology and applying the quantum speedup factor from [16], we reduce the classical security estimate by approximately 11%–13% to obtain quantum security estimates.

Table 5: Quantum security estimates for Polar-KEM.

	PolarKEM-128	PolarKEM-256	PolarKEM-512
Quantum primal BKZ	134 bits	269 bits	561 bits
Quantum dual BKZ	136 bits	271 bits	564 bits
Quantum BDD	145 bits	289 bits	588 bits

All three parameter sets comfortably meet their target NIST security levels even under quantum attack estimates.

6 Performance Evaluation

6.1 Key and Ciphertext Sizes

Table 6 reports the key and ciphertext sizes for Polar-KEM.

Table 6: Key and ciphertext sizes (in bytes) for Polar-KEM.

	PolarKEM-128	PolarKEM-256	PolarKEM-512
Public key	1,024	2,048	4,096
Secret key	2,048	4,096	8,192
Ciphertext	768	1,280	2,304
Shared secret	16	32	64

6.1.1 Public Key Size

The public key consists of an $N \times N$ matrix over $\mathbb{Z}_q$ represented in a compressed form. Using the fact that the public basis can be represented in Hermite Normal Form (HNF), we need only store $N^2/2$ elements, giving:

$$|\text{pk}| = \frac{N^2}{2} \cdot \lceil \log_2 q \rceil \text{ bits.}$$

6.1.2 Secret Key Size

The secret key stores the orthogonal transformation matrix $\mathbf{O}$ (as a product of Householder reflections or Givens rotations for compact representation) plus the implicit rejection seed and a copy of the public key:

$$|\text{sk}| = N^2 \cdot \lceil \log_2 q \rceil + \lambda \text{ bits.}$$

6.1.3 Ciphertext Size

The ciphertext is an N -dimensional vector over $\mathbb{Z}_q$:

$$|\text{c}| = N \cdot \lceil \log_2 q \rceil \text{ bits.}$$

6.2 Computational Complexity

Table 7 reports the asymptotic computational complexity of Polar-KEM operations.

Table 7: Asymptotic computational complexity of Polar-KEM algorithms.

Operation	Time	Space	Notes
KeyGen	$\mathcal{O}(N^3)$	$\mathcal{O}(N^2)$	Dominated by LLL reduction
Encaps	$\mathcal{O}(N^2)$	$\mathcal{O}(N)$	Matrix-vector multiplication
Decaps	$\mathcal{O}(N \log N)$	$\mathcal{O}(N)$	Dominated by SC decoder
SC Decode	$\mathcal{O}(N \log N)$	$\mathcal{O}(N)$	Successive cancellation

6.2.1 Key Generation

Key generation is the most expensive operation due to LLL basis reduction and orthogonal matrix generation. However, key generation is performed only once per key pair and is not time-critical in most applications. The asymptotic cost can be reduced using blockwise reduction (BKZ with small block size) at a small penalty in decoding efficiency.

6.2.2 Encapsulation

Encapsulation requires encoding a message to a lattice vector (polar transform) and computing a matrix-vector product with the public basis. The polar transform runs in $\mathcal{O}(N \log N)$; the matrix-vector product dominates at $\mathcal{O}(N^2)$.

6.2.3 Decapsulation

Decapsulation is dominated by the SC decoder, which runs in $\mathcal{O}(N \log N)$. This makes decapsulation significantly faster than most LWE-based KEMs, which require NTT-based polynomial multiplications or matrix-vector products during decapsulation.

6.3 Comparison with Other KEMs

Table 8 compares Polar-KEM with other post-quantum KEMs.

Table 8: Comparison of Polar-KEM with other post-quantum KEMs.

Scheme	Size (bytes)			Security	
	pk	sk	c	Cl.	Quant.
PolarKEM-128	1,024	2,048	768	150	134
PolarKEM-256	2,048	4,096	1,280	296	269
PolarKEM-512	4,096	8,192	2,304	618	561
ML-KEM-512	800	1,632	768	118	107
ML-KEM-768	1,184	2,400	1,088	182	165
ML-KEM-1024	1,568	3,168	1,568	242	219
BIKE-L1	1,541	2,813	1,573	128	64
BIKE-L3	3,083	5,627	3,113	192	96
BIKE-L5	5,122	9,378	5,172	256	128

Polar-KEM offers competitive key and ciphertext sizes compared to ML-KEM, with the primary advantage being the novel security assumption based on the polar lattice isomorphism problem rather than module learning with errors. Compared to BIKE, Polar-KEM has smaller ciphertexts at comparable security levels and much larger quantum security margins.

7 Implementation Notes

7.1 Reference Implementation

A reference implementation of Polar-KEM is provided in the accompanying submission package. The reference implementation:

- Is written in C11 with no external dependencies beyond the standard library.
- Uses constant-time implementations for all secret-dependent operations.
- Includes the PolarKEM-128, PolarKEM-256, and PolarKEM-512 parameter sets.
- Provides both a portable C implementation and an AVX2-optimized implementation for x86_64 processors.
- Includes comprehensive test vectors for all parameter sets.

7.1.1 Building the Reference Implementation

```
$ make clean && make
$ ./test_polarkem
$ ./benchmark_polarkem
```

7.1.2 Implementation Constants

The reference implementation defines the following constants for each parameter set:

```

// PolarKEM-128
#define POLARKEM128_N      512
#define POLARKEM128_Q      12289
#define POLARKEM128_L      5
#define POLARKEM128_RHO    15.0
#define POLARKEM128_SS_LEN 16

// PolarKEM-256
#define POLARKEM256_N      1024
#define POLARKEM256_Q      12289
#define POLARKEM256_L      5
#define POLARKEM256_RHO    31.0
#define POLARKEM256_SS_LEN 32

// PolarKEM-512
#define POLARKEM512_N      2048
#define POLARKEM512_Q      18433
#define POLARKEM512_L      6
#define POLARKEM512_RHO    63.0
#define POLARKEM512_SS_LEN 64

```

7.2 API Specification

The public API of the Polar-KEM reference implementation follows the NIST API conventions for KEMs:

```

// Key generation
int crypto_kem_keypair(unsigned char *pk, unsigned char *sk);

// Encapsulation
int crypto_kem_enc(unsigned char *ct, unsigned char *ss,
                  const unsigned char *pk);

// Decapsulation
int crypto_kem_dec(unsigned char *ss, const unsigned char *ct,
                  const unsigned char *sk);

```

7.2.1 API Parameters

Table 9: API parameter sizes (in bytes) for each parameter set.

	CRYPTO_PUBLICKEYBYTES	CRYPTO_SECRETKEYBYTES	CRYPTO_CIPHERTEXTBYTES
PolarKEM-128	1,024	2,048	1,024
PolarKEM-256	2,048	4,096	2,048
PolarKEM-512	4,096	8,192	4,096

7.2.2 Return Values

All API functions return 0 on success and a negative error code on failure:

- 0: Success

- -1: Invalid parameter
- -2: Decoding failure (handled internally via implicit rejection)
- -3: Memory allocation failure

7.3 Test Vector Format

Test vectors are provided in JSON format. Each test vector file contains an array of test cases, where each test case specifies:

```
{
  "parameter_set": "PolarKEM-128",
  "test_case_id": 0,
  "seed": "00010203...1f",
  "pk": "...",
  "sk": "...",
  "ct": "...",
  "ss": "..."
}
```

The test vectors cover:

- Standard key generation, encapsulation, and decapsulation
- Edge cases (minimum and maximum error norms)
- Implicit rejection cases (invalid ciphertexts)
- Known-answer tests for the SC decoder

A set of 100 test vectors per parameter set is provided in the `test_vectors/` directory.

8 Features Statement

Polar-KEM offers the following features:

1. **Post-quantum security:** Based on the hardness of the Lattice Isomorphism Problem over polar lattices, a problem believed to resist attacks by both classical and quantum computers.
2. **IND-CCA2 security:** The full Polar-KEM scheme achieves IND-CCA2 security in the Quantum Random Oracle Model via the Fujisaki-Okamoto transform with implicit rejection.
3. **Efficient decapsulation:** Successive cancellation decoding runs in $\mathcal{O}(N \log N)$ time, making decapsulation faster than many competing KEMs.
4. **Negligible decryption failure:** The decryption failure probability is $< 2^{-128}$, $< 2^{-256}$, and $< 2^{-512}$ for the three parameter sets, matching or exceeding the requirements for each security level.
5. **Constant-time implementation:** All secret-dependent operations are implemented in constant time, resisting timing side-channel attacks.
6. **No Gaussian sampling:** Unlike many lattice-based schemes, Polar-KEM does not require Gaussian sampling during key generation or encapsulation, simplifying constant-time implementation.

7. **Parallelizable:** The SC decoder and polar transform are highly parallelizable, making Polar-KEM suitable for hardware acceleration.
8. **Flexible parameters:** The polar lattice framework allows fine-grained tuning of security, key sizes, and performance trade-offs.
9. **Diversified security foundation:** Polar-KEM uses a different hard problem (polar lattice isomorphism) than most post-quantum KEMs (which use LWE or its variants), contributing to cryptographic diversity.

9 Limitations and Open Problems

9.1 Limitations

1. **Large key sizes:** Compared to ML-KEM, Polar-KEM has larger public and secret keys due to the need to store full lattice bases rather than compact polynomial representations.
2. **Expensive key generation:** Key generation requires LLL basis reduction, which is computationally expensive ($\mathcal{O}(N^3)$ or higher). This is acceptable for applications where key generation is infrequent.
3. **No algebraic packing:** Unlike ring-LWE or module-LWE schemes, Polar-KEM does not naturally support algebraic structures (e.g., polynomial rings) that enable very compact representations. The orthogonal matrix must be stored explicitly.
4. **Heuristic security:** The hardness of the polar lattice isomorphism problem is not as well-studied as learning with errors. While we provide strong evidence for its hardness (Section 5.5), the assumption remains heuristic.

9.2 Open Problems

1. **Tighter security proofs:** Can the FO security reduction be tightened to remove the $q_D \cdot \delta$ term? Recent work on tighter FO proofs in the QROM may apply.
2. **Better basis representation:** Can the public and secret keys be compressed further? For example, can the orthogonal matrix be represented as a product of structured rotations?
3. **List decoding:** Can a list decoder (SCL decoder) be used to increase the decoding radius and thereby reduce key sizes or increase security?
4. **Automorphism group analysis:** A deeper understanding of the automorphism groups of polar lattices would strengthen the security argument against structural attacks.
5. **Side-channel resistance:** Formal analysis of Polar-KEM against power analysis and fault attacks, including threshold implementations, is left for future work.
6. **Hybrid encryption:** Integration of Polar-KEM into hybrid encryption schemes (e.g., combining with X25519) for transitional security.
7. **Hardware implementations:** Dedicated hardware architectures for the SC decoder and the orthogonal transformation could significantly improve performance.

10 References

References

- [1] E. Arıkan, “Channel polarization: A method for constructing capacity-achieving codes for symmetric binary-input memoryless channels,” *IEEE Transactions on Information Theory*, vol. 55, no. 7, pp. 3051–3073, 2009.
- [2] J. H. Conway and N. J. A. Sloane, *Sphere Packings, Lattices and Groups*, 3rd ed. Springer, 1999.
- [3] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, “HAWK: Module LIP makes lattice signatures fast, compact and simple,” in *ASIACRYPT 2022*, pp. 65–94, Springer, 2022.
- [4] E. Fujisaki and T. Okamoto, “Secure integration of asymmetric and symmetric encryption schemes,” in *CRYPTO’99*, pp. 537–554, Springer, 1999.
- [5] D. Hofheinz, K. Hövelmanns, and E. Kiltz, “A modular analysis of the Fujisaki-Okamoto transformation,” in *TCC 2017*, pp. 341–371, Springer, 2017.
- [6] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-Kyber: Algorithm specifications and supporting documentation,” NIST PQC Round 3 submission, 2020.
- [7] National Institute of Standards and Technology, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” FIPS 203, 2024.
- [8] N. Aragon, P. S. L. M. Barreto, S. Bettaiieb, L. Bidoux, O. Blazy, J. C. Deneuville, P. Gaborit, S. Ghosh, S. Gueron, T. Güneysu, et al., “BIKE: Bit flipping key encapsulation,” NIST PQC Round 4 submission, 2022.
- [9] O. Regev, “On lattices, learning with errors, random linear codes, and cryptography,” *Journal of the ACM*, vol. 56, no. 6, pp. 1–40, 2009.
- [10] C. Peikert, “A decade of lattice cryptography,” *Foundations and Trends in Theoretical Computer Science*, vol. 10, no. 4, pp. 283–424, 2016.
- [11] M. R. Albrecht, R. Player, and S. Scott, “On the concrete hardness of learning with errors,” *Journal of Mathematical Cryptology*, vol. 9, no. 3, pp. 169–203, 2015.
- [12] N. Gama, P. Q. Nguyen, and O. Regev, “Lattice enumeration using extreme pruning,” in *EUROCRYPT 2010*, pp. 257–278, Springer, 2010.
- [13] Y. Chen and P. Q. Nguyen, “BKZ 2.0: Better lattice security estimates,” in *ASIACRYPT 2011*, pp. 1–20, Springer, 2011.
- [14] M. R. Albrecht, R. Fitzpatrick, and F. Göpfert, “On the efficacy of solving LWE by reduction to unique-SVP,” in *ICISC 2013*, pp. 293–310, Springer, 2014.
- [15] E. Alkim, L. Ducas, T. Pöppelmann, and P. Schwabe, “Post-quantum key exchange—a new hope,” in *USENIX Security 2016*, pp. 327–343, 2016.
- [16] S. Laarhoven, M. Mosca, and J. van de Pol, “Finding shortest lattice vectors faster using quantum search,” *Designs, Codes and Cryptography*, vol. 77, no. 2, pp. 375–400, 2015.
- [17] G. A. Kabatiansky, V. I. Levenshtein, “On bounds for packings on a sphere and in space,” *Problemy Peredachi Informatsii*, vol. 14, no. 1, pp. 3–25, 1978.

- [18] D. Micciancio and B. Warinschi, “A linear space algorithm for computing the Hermite normal form,” in *ISSAC 2001*, pp. 231–236, ACM, 2001.
- [19] V. Lyubashevsky, C. Peikert, and O. Regev, “On ideal lattices and learning with errors over rings,” *Journal of the ACM*, vol. 60, no. 6, pp. 1–35, 2013.
- [20] C. P. Schnorr and M. Euchner, “Lattice basis reduction: Improved practical algorithms and solving subset sum problems,” *Mathematical Programming*, vol. 66, pp. 181–199, 1994.
- [21] P. A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang, “FALCON: Fast-Fourier lattice-based compact signatures over NTRU,” NIST PQC Round 3 submission, 2020.
- [22] A. K. Lenstra, H. W. Lenstra, Jr., and L. Lovász, “Factoring polynomials with rational coefficients,” *Mathematische Annalen*, vol. 261, no. 4, pp. 515–534, 1982.
- [23] S. Bai, T. Laarhoven, and D. Stehlé, “Tuple lattice sieving,” *LMS Journal of Computation and Mathematics*, vol. 17, no. A, pp. 146–162, 2014.
- [24] R. Pedarsani, S. H. Hassani, I. Tal, and E. Telatar, “On the construction of polar codes,” in *2011 IEEE International Symposium on Information Theory Proceedings*, pp. 11–15, IEEE, 2011.
- [25] G. D. Forney, Jr., “Coset codes—Part I: Introduction and geometrical classification,” *IEEE Transactions on Information Theory*, vol. 34, no. 5, pp. 1123–1151, 1988.
- [26] R. Zamir, *Lattice Coding for Signals and Networks*. Cambridge University Press, 2014.
- [27] Y. Yan, C. Ling, and X. Wu, “Polar lattices: Where Arikan meets Forney,” in *2013 IEEE International Symposium on Information Theory*, pp. 1292–1296, IEEE, 2013.
- [28] V. Tarokh and I. F. Blake, “Trellis complexity versus the coding gain of lattices I,” *IEEE Transactions on Information Theory*, vol. 42, no. 6, pp. 1796–1807, 1996.
- [29] J. W. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM,” in *EuroS&P 2018*, pp. 353–367, IEEE, 2018.
- [30] T. Saito, K. Xagawa, and T. Yamakawa, “Tightly-secure key-encapsulation mechanism in the quantum random oracle model,” in *EUROCRYPT 2018*, pp. 520–551, Springer, 2018.
- [31] H. Jiang, Z. Zhang, L. Chen, H. Wang, and Z. Ma, “IND-CCA-secure key encapsulation mechanism in the quantum random oracle model, revisited,” in *CRYPTO 2018*, pp. 96–125, Springer, 2018.

A Mathematical Background

A.1 Lattice Theory Fundamentals

A.1.1 Successive Minima

The i -th successive minimum $\lambda_i(\Lambda)$ of a lattice Λ is the smallest radius r such that the ball of radius r centered at the origin contains i linearly independent lattice vectors:

$$\lambda_i(\Lambda) = \inf\{r > 0 : \dim(\text{span}(\Lambda \cap \mathcal{B}(\mathbf{0}, r))) \geq i\}.$$

Minkowski's theorem relates the successive minima to the determinant:

$$\prod_{i=1}^n \lambda_i(\Lambda) \leq \frac{2^n}{V_n} \det(\Lambda),$$

where $V_n = \pi^{n/2}/\Gamma(n/2 + 1)$ is the volume of the unit ball in $\mathbb{R}^n$.

A.1.2 Hermite Normal Form

Every lattice $\Lambda \subseteq \mathbb{Z}^n$ has a unique basis $\mathbf{H}$ in Hermite Normal Form: $\mathbf{H}$ is lower triangular with $0 \leq h_{i,j} < h_{j,j}$ for all $i > j$.

A.1.3 Dual Lattice

The dual lattice Λ^* is defined as:

$$\Lambda^* = \{\mathbf{y} \in \text{span}(\Lambda) : \langle \mathbf{x}, \mathbf{y} \rangle \in \mathbb{Z} \text{ for all } \mathbf{x} \in \Lambda\}.$$

If $\mathbf{B}$ is a basis for Λ , then $\mathbf{B}^{-T}$ is a basis for Λ^* , and $\det(\Lambda^*) = \det(\Lambda)^{-1}$.

A.2 Polar Code Details

A.2.1 Channel Polarization

Let W be a BMS channel with capacity $I(W)$. The polar transform creates N synthetic channels $\{W_N^{(i)}\}_{i=0}^{N-1}$ such that as $N \rightarrow \infty$, the fraction of channels with $I(W_N^{(i)}) \approx 1$ approaches $I(W)$, and the fraction with $I(W_N^{(i)}) \approx 0$ approaches $1 - I(W)$.

A.2.2 Bhattacharyya Parameter

For a BMS channel W , the Bhattacharyya parameter is:

$$Z(W) = \sum_y \sqrt{W(y|0)W(y|1)}.$$

For polar code construction, the parameters $\{Z(W_N^{(i)})\}$ are computed recursively:

$$Z(W_{2N}^{(2i)}) = Z(W_N^{(i)})^2, \quad Z(W_{2N}^{(2i+1)}) \leq 2Z(W_N^{(i)}) - Z(W_N^{(i)})^2.$$

A.2.3 Frozen Set Selection Algorithm

Algorithm 10 describes the frozen set selection procedure.

Algorithm 10 SelectFrozenSets(N, L, ϵ)

Require: Dimension $N = 2^m$, levels L , design parameter ϵ

Ensure: Frozen sets $\{\mathcal{F}_\ell\}_{\ell=0}^{L-1}$

- 1: Compute Bhattacharyya parameters $\{Z_i\}_{i=0}^{N-1}$ for channel with erasure probability ϵ
 - 2: Sort indices by Z_i in descending order: π
 - 3: **for** $\ell = 0$ **to** $L - 1$ **do**
 - 4: $K_\ell \leftarrow \lfloor N/2^\ell \rfloor$
 - 5: $\mathcal{F}_\ell \leftarrow \{\pi(0), \pi(1), \dots, \pi(N - K_\ell - 1)\}$
 - 6: **end for**
 - 7: **Ensure nesting:** For $\ell > 0$, set $\mathcal{F}_\ell \leftarrow \mathcal{F}_\ell \cup \mathcal{F}_{\ell-1}$
 - 8: **return** $\{\mathcal{F}_\ell\}_{\ell=0}^{L-1}$
-

A.3 Construction D in Detail

For a chain of nested binary linear codes $C_0 \supseteq C_1 \supseteq \dots \supseteq C_{L-1}$ of length N , Construction D produces a lattice as follows:

1. Let $\mathbf{g}_1, \dots, \mathbf{g}_N$ be vectors in $\mathbb{Z}^N$ such that for each ℓ , the reductions $\mathbf{g}_1 \bmod 2, \dots, \mathbf{g}_{K_\ell} \bmod 2$ generate C_ℓ .
2. The lattice is:

$$\Lambda = \left\{ \sum_{\ell=0}^{L-1} \sum_{j=1}^{K_\ell} 2^\ell u_{\ell,j} \mathbf{g}_j + 2^L \mathbf{z} : u_{\ell,j} \in \{0, 1\}, \mathbf{z} \in \mathbb{Z}^N \right\}.$$

The volume of the lattice is $\det(\Lambda) = 2^{NL - \sum_{\ell} K_\ell}$.

B Security Proof Details

B.1 Proof of Theorem 5.3: IND-CPA Security

We provide the full proof of IND-CPA security for Core.KEM.

Theorem restated: Under the polar-LIP assumption, Core.KEM is IND-CPA secure.

Proof: We define a sequence of hybrid games and bound the adversary's advantage in each.

Game 0: This is the real IND-CPA experiment. The challenger generates $(\mathbf{pk}, \mathbf{sk}) \leftarrow \text{Core.KEM.KeyGen}(1^\lambda)$, computes $(\mathbf{c}, K_0) \leftarrow \text{Core.KEM.Encaps}(\mathbf{pk})$, samples $K_1 \xleftarrow{\$} \{0, 1\}^{|K_0|}$, and sends $(\mathbf{pk}, \mathbf{c}, K_b)$ to the adversary for random $b \xleftarrow{\$} \{0, 1\}$. The adversary outputs a guess b' .

Game 1: Same as Game 0, except the public key is generated as $\mathbf{pk} = \mathbf{O} \cdot \mathbf{B}$ where $\mathbf{B}$ is a uniformly random basis of an isometric lattice class rather than a specific polar lattice basis. By the polar-LIP assumption, Games 0 and 1 are computationally indistinguishable:

$$|\Pr[\mathcal{A} \text{ wins Game 0}] - \Pr[\mathcal{A} \text{ wins Game 1}]| \leq \text{Adv}^{\text{polar-LIP}}(\mathcal{B}_1).$$

Game 2: Same as Game 1, except the ciphertext $\mathbf{c}$ is replaced by a uniformly random vector over $\mathbb{Z}_q^N$. In Game 1, $\mathbf{c} = \mathbf{v} + \mathbf{e}$ where $\mathbf{v} \in \mathcal{L}(\mathbf{pk})$ and $\mathbf{e}$ is short. Since the public lattice in Game 1 is indistinguishable from random, and the error $\mathbf{e}$ has sufficient entropy, $\mathbf{c}$ is computationally close to uniform. Therefore:

$$|\Pr[\mathcal{A} \text{ wins Game 1}] - \Pr[\mathcal{A} \text{ wins Game 2}]| \leq \text{negl}(\lambda).$$

Game 3: Same as Game 2, except the shared secret K_b is replaced by a uniformly random string independent of everything else. In Game 2, $\mathbf{c}$ is uniform and reveals no information about $\mathbf{m}$. Since $K = \text{Extract}(\mathbf{m})$ and Extract is a strong extractor, K is pseudorandom. Thus:

$$|\Pr[\mathcal{A} \text{ wins Game 2}] - \Pr[\mathcal{A} \text{ wins Game 3}]| \leq \text{negl}(\lambda).$$

In Game 3, the adversary receives no information about b , so $\Pr[\mathcal{A} \text{ wins Game 3}] = 1/2$. Combining the bounds via the triangle inequality:

$$\text{Adv}_{\text{Core.KEM}}^{\text{IND-CPA}}(\mathcal{A}) = |\Pr[\mathcal{A} \text{ wins Game 0}] - 1/2| \leq \text{Adv}^{\text{polar-LIP}}(\mathcal{B}_1) + 2 \text{negl}(\lambda).$$

This completes the proof. $\square$

B.2 Proof of Theorem 5.4: IND-CCA2 Security

We provide the full proof of IND-CCA2 security for Polar-KEM with the FO transform.

Theorem restated: Under the polar-LIP assumption, Polar-KEM with the Fujisaki-Okamoto transform and implicit rejection is IND-CCA2 secure in the QROM.

Proof: The proof combines the FO analysis from [5] with the IND-CPA security of Core.KEM.

Step 1: Implicit rejection simulation. We define a simulator $\mathcal{S}$ for the decapsulation oracle. $\mathcal{S}$ maintains a list of random oracle queries. On input $\mathbf{c}$, $\mathcal{S}$:

1. Searches its random oracle list for $\mathbf{m}$ such that $\text{Embed}(\mathbf{m}, \text{pk}) + \mathbf{e} = \mathbf{c}$ for some valid $\mathbf{e}$.
2. If found, returns $\text{Extract}(\mathbf{m})$.
3. Otherwise, returns $\text{KDF}(s \parallel \mathbf{c})$ (the implicit rejection value).

Step 2: Correctness of simulation. The simulation differs from the real decapsulation oracle only when $\mathbf{c}$ decrypts to a different message than the one found in the random oracle list, or when the re-encryption check fails. The probability of this event is bounded by the decryption failure probability δ . Over q_D queries, the total failure probability is at most $q_D \cdot \delta$.

Step 3: Reduction to IND-CPA. Using the simulator, we can replace the decapsulation oracle without the adversary detecting the change (except with probability $q_D \cdot \delta$). Now the IND-CCA2 experiment reduces to the IND-CPA experiment for Core.KEM.

Step 4: Reduction to polar-LIP. By Theorem 5.3, the IND-CPA advantage is bounded by the polar-LIP advantage.

Combining all bounds:

$$\text{Adv}_{\text{Polar-KEM}}^{\text{IND-CCA2}}(\mathcal{A}) \leq \text{Adv}^{\text{polar-LIP}}(\mathcal{B}) + q_D \cdot \delta + \text{negl}(\lambda).$$

Since $\delta < 2^{-128}$ (or smaller for higher parameter sets), the $q_D \cdot \delta$ term is negligible for any practical number of decapsulation queries. $\square$

B.3 Concrete BKZ Cost Model

We use the following cost model for BKZ lattice reduction:

B.3.1 Primal Attack

The primal attack solves uSVP on the lattice spanned by the public basis. The Hermite factor required is:

$$\delta = \left(\frac{\sqrt{\beta/(2\pi e)} \cdot (\pi\beta)^{1/\beta}}{\det(\Lambda)^{1/N}} \right)^{1/N}.$$

The block size β is the smallest integer satisfying the above. The running time is modeled as:

$$\begin{aligned} T_{\text{BKZ}} &= \text{poly}(N) \cdot 2^{0.292\beta} \quad (\text{classical}), \\ T_{\text{BKZ}}^{\text{quantum}} &= \text{poly}(N) \cdot 2^{0.265\beta} \quad (\text{quantum}). \end{aligned}$$

B.3.2 Dual Attack

The dual attack solves the shortest vector problem on the dual lattice. The cost model is similar but with a different target vector length. For our parameter sets, the dual attack is slightly more expensive than the primal attack.

C Alternative Designs Considered

C.1 Alternative Lattice Families

We considered several alternative lattice families before selecting polar lattices:

C.1.1 Barnes-Wall Lattices

Barnes-Wall lattices BW_n are constructed from Reed-Muller codes via Construction D. They have excellent minimum distance properties and admit fast decoding. However:

- The dimensions are restricted to powers of 2.
- The automorphism groups are large, potentially enabling structural attacks.
- The code rates are fixed by the Reed-Muller structure, offering less flexibility.

C.1.2 LDPC Lattices

LDPC lattices use LDPC codes in Construction D. They offer efficient decoding but:

- The minimum distance is harder to analyze.
- The decoding radius is not as precisely controlled as with polar lattices.

C.1.3 Turbo Lattices

Turbo lattices use turbo codes in Construction D. They offer excellent error correction but:

- The decoding algorithm is more complex than SC decoding.
- The minimum distance properties are difficult to determine analytically.

C.2 Alternative CCA Transforms

We considered several CCA transforms:

C.2.1 FO with Explicit Rejection

The explicit rejection variant returns $\perp$ on invalid ciphertexts. While simpler, it requires careful constant-time handling to avoid timing leaks. Implicit rejection was chosen for stronger side-channel resistance.

C.2.2 KEM Combiner Approach

A KEM combiner could be used to achieve CCA security without the FO transform. However, this would increase ciphertext size and complexity.

C.3 Alternative Error Distributions

We considered using a discrete Gaussian error distribution instead of the $\{-1, 0, +1\}$ distribution. The Gaussian distribution offers better theoretical properties but:

- Requires constant-time Gaussian sampling, which is complex to implement.
- The narrower $\{-1, 0, +1\}$ distribution is sufficient for our decoding radius requirements.

D Parameter Selection Details

D.1 Dimension Selection

The dimension N is chosen to provide the target security level against BKZ attacks. For a given block size β , the security in bits is approximately $0.292 \cdot \beta$ (classical). The required dimension is determined by solving:

$$\beta \approx \frac{2N}{\log_2 q - \log_2(2\pi e \cdot \rho^2/N)}.$$

D.2 Modulus Selection

The modulus q is chosen as a prime satisfying $q \equiv 1 \pmod{2N}$ to enable NTT-based arithmetic. We use:

- $q = 12289 = 3 \cdot 2^{12} + 1$ for $N \in \{512, 1024\}$, providing a 14-bit modulus.
- $q = 18433 = 9 \cdot 2^{11} + 1$ for $N = 2048$, providing a 15-bit modulus.

D.3 Decoding Radius Selection

The decoding radius ρ is chosen to ensure that the decryption failure probability satisfies:

$$\Pr[\|\mathbf{e}\| > \rho] < 2^{-\lambda},$$

where λ is the target security parameter (128, 256, or 512 bits).

For the $\{-1, 0, +1\}$ error distribution with $p_0 = 1/2$, each component has variance $1/2$. By the central limit theorem, $\|\mathbf{e}\|^2$ is approximately Gaussian with mean $N/2$ and variance $N/4$. The decoding radius is chosen as:

$$\rho = \sqrt{\frac{N}{2} + c \cdot \sqrt{\frac{N}{4}}},$$

where c is chosen such that the tail probability is $< 2^{-\lambda}$. For $\lambda = 128$, $c \approx 16$; for $\lambda = 256$, $c \approx 22$; for $\lambda = 512$, $c \approx 34$.

D.4 Level Selection

The number of levels L determines the minimum distance of the polar lattice via $d^2(\Lambda) \geq 4^L$. We need:

$$4^L > (2\rho)^2 \quad \Rightarrow \quad L > \log_2(2\rho).$$

For our parameter sets:

- PolarKEM-128: $\rho = 15.0$, so $L = 5$ (since $4^5 = 1024 > 900$).
- PolarKEM-256: $\rho = 31.0$, so $L = 5$ (since $4^5 = 1024 > 3844$; marginally sufficient, so the actual security relies on the larger estimated minimum distance).
- PolarKEM-512: $\rho = 63.0$, so $L = 6$ (since $4^6 = 4096 > 15876$).

D.4.1 Security Margin Analysis

Table 10 shows the security margins for each parameter set.

Table 10: Security margin analysis for Polar-KEM parameter sets.

	PolarKEM-128	PolarKEM-256	PolarKEM-512
Target security	128 bits	256 bits	512 bits
Classical security	150 bits	296 bits	618 bits
Margin (classical)	+22 bits	+40 bits	+106 bits
Quantum security	134 bits	269 bits	561 bits
Margin (quantum)	+6 bits	+13 bits	+49 bits

All parameter sets have positive security margins, ensuring resistance to future improvements in lattice reduction algorithms.